

Finite Difference Method for the Reaction Heat/Diffusion Equation

Erik Brobyn

Introduction

Reaction Heat/Diffusion Equation

$$\frac{\partial C}{\partial t} = a \frac{\partial^2 C}{\partial x^2} - kC \quad (1)$$

Initial Conditions (IC)

$$C(x, 0) = \begin{cases} 1 & |x| < 1 \\ 0 & |x| > 1 \end{cases} \quad (2)$$

Note these conditions are symmetric about x .

The governing partial differential equation (PDE) for this report, the Reaction Heat/Diffusion Equation (1), and the classic engineering problem surrounding it, predicts how substances spread out and disappear over time. For example, we could call $C(x, t)$ the Scalar Pollutant Concentration. Start with a localized amount dumped into an infinite space at time zero. Two physical competing forces take over: diffusion (a) flattens the sharp edges, and reaction (k) decays the concentration everywhere.

This report focuses on Finite Difference Methods (FDM), the popular solvers used for approximating (1) with ICs (2) in a computational environment, and in particular, the Forward-Time Central-Space (FTCS) Method. In this report, FTCS is the FDM benchmark used for comparisons, and its stencil (3) is derived directly from the Taylor Series Expansion. Some other intriguing methodologies also approximate a solution, and are discussed in greater detail later on in the survey.

FTCS: Benchmark FDM Stencil

$$C_i^{n+1} = C_i^n + \Delta t \left[a \frac{C_{i+1}^n - 2C_i^n + C_{i-1}^n}{(\Delta x)^2} - kC_i^n \right] \quad (3)$$

1	Analytic Solution	2
2	Benchmark Stencil Derivation	6
3	Benchmark Comparison with Alternative FDM Stencils	8
4	A General Survey of Solvers	10
5	FDM Performance Comparison with Python	14
6	Benchmark Implementation Comparison (Python, OpenMPI, CUDA)	

1 Analytic Solution

Consider the following mathematical concepts related to this problem, the Laplacian (∇^2) operator and the general Heat/Diffusion Equation:

The Laplacian ∇^2

Generally, the Laplacian measures how a quantity diffuses or spreads out from a point, and is normally a 3-D spatial operator. In this case, that simplifies down to just the second derivative along x :

$$\nabla^2 C = \frac{\partial^2 C}{\partial x^2} \quad (4)$$

Since ICs (2) are symmetric, as is (4), the solution must also be symmetric.

Heat/Diffusion Equation

(1) is a special case of the more general 1-D Heat/Diffusion Equation.

$$\frac{\partial u}{\partial t} = a \frac{\partial^2 u}{\partial x^2} \quad (5)$$

To find it, introduce a transformation to remove the $-kC$ term:

$$C(x, t) = e^{-kt} u(x, t) \quad (6)$$

Differentiate the expression with respect to t and x :

$$\begin{aligned} \frac{\partial C}{\partial t} &= -ke^{-kt}u + e^{-kt}\frac{\partial u}{\partial t} \\ \frac{\partial^2 C}{\partial x^2} &= e^{-kt}\frac{\partial^2 u}{\partial x^2} \end{aligned}$$

Substitute those results back into (1):

$$-ke^{-kt}u + e^{-kt}\frac{\partial u}{\partial t} = ae^{-kt}\frac{\partial^2 u}{\partial x^2} - ke^{-kt}u$$

To complete the transformation to (5), cancel the matching reaction terms and divide by e^{-kt} . Since $e^0 = 1$, the transformed initial conditions at $t = 0$ remain unchanged:

$$u(x, 0) = \begin{cases} 1 & |x| < 1 \\ 0 & |x| > 1 \end{cases}$$

Separation of Variables

When we use the separation of variables method on the Heat/Diffusion equation, we separate the time part from the space part. That is, if we first assume $u(x, t) = X(x)T(t)$, then

$$X(x)T'(t) = aX''(x)T(t) \implies \frac{T'}{aT} = \frac{X''}{X} = -\lambda^2$$

Notice that, by convention, we chose $-\lambda^2$ to be negative. This yields two ordinary differential equations (ODEs), the time equation:

$$T' + a\lambda^2 T = 0 \implies T(t) = e^{-a\lambda^2 t}$$

and the space, or 1-D Helmholtz equation:

$$X'' = -\lambda^2 X \implies X(x) = A \cos(\lambda x) + B \sin(\lambda x) \quad (7)$$

Note that $-\lambda^2$ has to be negative because otherwise our time function would blow up for $t \rightarrow \infty$. Therefore, the associated spatial function has to be oscillatory (sine, cosine or exponential $e^{\pm ikz}$), and the time function must be an exponential decay (hyperbolic sine, cosine, or e^{-kz}). However, if we had chosen $+\lambda^2$, we would invert our spatial solution to a combination of hyperbolic cosine and sine, and use exponential growth (e^{+kt}) for the time function.

Helmholtz Equation

The 1-D Helmholtz equation is:

$$\frac{d^2 X}{dx^2} + \lambda^2 X = 0 \quad (8)$$

The Helmholtz equation determines the spatial shapes or modes (sines and cosines) that the substance takes as it spreads. Because in this case the ICs are symmetric, the sine terms in (7) drop out ($B = 0$).

Fourier Transform

Since the domain is infinite ($-\infty < x < \infty$), λ must be continuous, transforming the solution for $u(x, t)$ into a Fourier integral:

$$u(x, t) = \int_0^\infty A(\lambda) \cos(\lambda x) e^{-a\lambda^2 t} d\lambda \quad (9)$$

Applying initial conditions at $t = 0$, we find:

$$u(x, 0) = \int_0^\infty A(\lambda) \cos(\lambda x) d\lambda$$

And evaluating the Fourier transform, $A(\lambda)$:

$$A(\lambda) = (2) \frac{1}{2\pi} \left[2 \int_0^\infty u(x, 0) \cos(\lambda x) dx \right] = \frac{2}{\pi} \int_0^1 \cos(\lambda x) dx = \left(\frac{2}{\pi} \right) \frac{\sin(\lambda x)}{\lambda} \Big|_0^1 = \frac{2 \sin(\lambda)}{\pi \lambda} \quad (10)$$

Normalization Analysis

Considering the results of (10), it makes sense that A include sine, the orthogonal partner to cosine, which arises from the symmetry requirements, and that A takes an argument produced by λx when $x = 1$. Note also that $\frac{\sin(\lambda)}{\lambda} \rightarrow 1$ as $\lambda \rightarrow 0$. Two factors of (2) appear in the Fourier transform due to the solution symmetry and doubling of half-space limits to positive x values.

Trigonometric Decomposition

Plug the results from (10) back into (9):

$$u(x, t) = \frac{2}{\pi} \int_0^\infty \frac{\sin(\lambda)}{\lambda} \cos(\lambda x) e^{-a\lambda^2 t} d\lambda \quad (11)$$

Decompose the trigonometric product using the product-to-sum identity:

$$2 \sin(\lambda) \cos(\lambda x) = \sin[\lambda(1 + x)] + \sin[\lambda(1 - x)]$$

Substituting that expression back into (11) splits the integral into two:

$$u(x, t) = \frac{1}{\pi} \int_0^\infty \left[\frac{\sin(\lambda(1 + x))}{\lambda} + \frac{\sin(\lambda(1 - x))}{\lambda} \right] e^{-a\lambda^2 t} d\lambda \quad (12)$$

Leibniz's Rule Trick

With the time-dependent frequency modes established, and without loss of generality, let:

$$\chi = 1 \pm x \quad (13)$$

Here the + or - depends on which integral is chosen from the above sum in (12).

Let's define two functions $I(\chi)$ and $J(\chi) = I'(\chi)$:

$$I(\chi) = \int_0^\infty \frac{\sin(\chi\lambda)}{\lambda} e^{-a\lambda^2 t} d\lambda \quad (14)$$

Using Leibniz's Rule, differentiating (14) with respect to χ eliminates λ from the denominator:

$$J(\chi) = \frac{dI}{d\chi} = \frac{d}{d\chi} \left[\int_0^\infty \frac{\sin(\chi\lambda)}{\lambda} e^{-a\lambda^2 t} d\lambda \right] = \int_0^\infty \frac{\partial}{\partial \chi} \left[\frac{\sin(\chi\lambda)}{\lambda} \right] e^{-a\lambda^2 t} d\lambda = \int_0^\infty \cos(\chi\lambda) e^{-a\lambda^2 t} d\lambda$$

Similarly, applying this rule once again recovers an integrable expression.

$$\frac{dJ}{d\chi} = \int_0^\infty \frac{d}{d\chi} [\cos(\chi\lambda)] e^{-a\lambda^2 t} d\lambda = - \int_0^\infty \lambda \sin(\chi\lambda) e^{-a\lambda^2 t} d\lambda$$

Integration by Parts

$$\int u dv = uv - \int v du$$

With substitutions $u = \sin(\chi\lambda)$ and $dv = -\lambda e^{-a\lambda^2 t} d\lambda$, we find:

$$\frac{dJ}{d\chi} = - \int_0^\infty \lambda \sin(\chi\lambda) e^{-a\lambda^2 t} d\lambda = \frac{1}{2at} \sin(\chi\lambda) e^{-a\lambda^2 t} \Big|_0^\infty - \int_0^\infty \frac{\chi}{2at} \cos(\chi\lambda) e^{-a\lambda^2 t} d\lambda = -\frac{\chi}{2at} J$$

Rearranging the expressions:

$$\frac{dJ}{J} = -\frac{1}{2at} \chi d\chi$$

Integrating both sides:

$$\int \frac{dJ}{J} = \ln J = -\frac{1}{2at} \int \chi d\chi = -\frac{\chi^2}{4at} + C$$

$$J(\chi) = Ge^{-\frac{\chi^2}{4at}}$$

Where $G = e^C$. Now, $J(0) = Ge^0 = G$, so

$$G = J(0) = \int_0^\infty \cos(0) e^{-a\lambda^2 t} d\lambda = \int_0^\infty e^{-a\lambda^2 t} d\lambda \quad (15)$$

Gaussian Integral

Thus, we arrive at (15), the famous Gaussian Integral. Start by squaring it. Note that the variables used inside the product of two integrals are always orthogonal to one another, and may therefore be transformed from the Cartesian to the cylindrical coordinate system.

$$G^2 = \int_0^\infty e^{-a\lambda_1^2 t} d\lambda_1 \int_0^\infty e^{-a\lambda_2^2 t} d\lambda_2 = \int_0^\infty \int_0^\infty e^{-a(\lambda_1^2 + \lambda_2^2)t} d\lambda_1 d\lambda_2 = \int_0^{\frac{\pi}{2}} d\phi \int_0^\infty r dr e^{-ar^2 t}$$

Noting the substitution $r^2 = \lambda_1^2 + \lambda_2^2$, and the limits λ_1, λ_2 from 0 to ∞ transform ϕ limits from 0 to $\frac{\pi}{2}$ (only the ++ quadrant of $\mathbb{R} \times \mathbb{R}$), and r from 0 to ∞ .

Substitute $u = ar^2t \implies du = a2rt dr$:

$$\int_0^{\frac{\pi}{2}} d\phi \int_0^\infty r dr e^{-ar^2t} = \frac{\pi}{2} \left(\frac{1}{2at} \right) \int_0^\infty e^{-u} du = \frac{\pi}{4at} (-1) e^{-u} \Big|_0^\infty \implies G = \sqrt{\frac{\pi}{4at}}$$

$$\frac{dI}{d\chi} = \sqrt{\frac{\pi}{4at}} e^{-\frac{\chi^2}{4at}}$$

Now, we recover the original function $I(\chi)$ by integrating this, using a new dummy variable ξ that ranges from 0 to χ :

$$\sqrt{\frac{\pi}{4at}} \int_0^\chi e^{-\frac{\xi^2}{4at}} d\xi = I(\chi) - I(0) \quad (16)$$

From the definition of $I(\chi)$, it's clear that if $\chi = 0$, $\sin(0) = 0$ will force the initial condition to zero:

$$I(0) = \int_0^\infty \frac{\sin(0)}{\lambda} e^{-a\lambda^2t} d\lambda = 0$$

Thus (16) reduces to:

$$I(\chi) = \sqrt{\frac{\pi}{4at}} \int_0^\chi e^{-\frac{\xi^2}{4at}} d\xi \quad (17)$$

Introducing erf

This is the Gaussian Error Function erf:

$$\text{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-\eta^2} d\eta \quad (18)$$

To use erf, starting from (17), introduce one more dummy variable, η , to clean up the exponent:

$$\eta = \frac{\xi}{2\sqrt{at}} \implies d\eta = \frac{d\xi}{2\sqrt{at}} \implies d\xi = 2\sqrt{at} d\eta$$

So when $\xi = \chi$, $\eta = \frac{\chi}{2\sqrt{at}}$. Substituting these variables back into the expression yields:

$$I(\chi) = \sqrt{\pi} \int_0^{\frac{\chi}{2\sqrt{at}}} e^{-\eta^2} d\eta$$

$$I(\chi) = \frac{\pi}{2} \left(\frac{2}{\sqrt{\pi}} \int_0^{\frac{\chi}{2\sqrt{at}}} e^{-\eta^2} d\eta \right) = \frac{\pi}{2} \text{erf} \left(\frac{\chi}{2\sqrt{at}} \right)$$

Final Analytic Solution

Substituting (13) back in for χ and into (12), we find:

$$u(x, t) = \frac{1}{\pi} \frac{\pi}{2} \left[\text{erf} \left(\frac{1+x}{2\sqrt{at}} \right) + \text{erf} \left(\frac{1-x}{2\sqrt{at}} \right) \right] \quad (19)$$

Reversing the initial transformation back into (6) to arrive at the final analytical solution:

$$\mathbf{C}(\mathbf{x}, \mathbf{t}) = \frac{1}{2} \mathbf{e}^{-\mathbf{k}\mathbf{t}} \left[\text{erf} \left(\frac{1+\mathbf{x}}{2\sqrt{\mathbf{a}\mathbf{t}}} \right) + \text{erf} \left(\frac{1-\mathbf{x}}{2\sqrt{\mathbf{a}\mathbf{t}}} \right) \right] \quad (20)$$

2 Benchmark Stencil Derivation

What follows is an analysis of the stenciling technique used to approximate the solution for (1) with ICs (2) using the standard Explicit FDM, Forward-Time-Central-Space (FTCS), and its derivation directly from the Taylor Series Expansion.

Let $C(x, t)$ be defined as in (20), the unique solution for (1) with ICs (2). Recall also that a is the diffusion coefficient ($a > 0$), and k is the reaction or decay coefficient ($k \geq 0$).

Domain Discretizations

To solve the equation numerically, we map the continuous domain onto a discrete grid with uniform spacing. Space is partitioned into N_x nodes with spacing Δx , indexed by i . Time is partitioned into snapshots with interval Δt , indexed by n .

Spatial step size: Δx , where $x_i = i\Delta x$

Temporal step size: Δt , where $t_n = n\Delta t$

Grid notation: $C(x_i, t_n) \approx C_i^n$

Taylor Series Expansion

The foundation of any finite difference method relies on approximating derivatives using a Taylor series expansion. If a function $f(x)$ is infinitely differentiable in the neighborhood of a point x_0 , the value of the function at a nearby point $x = x_0 + \Delta x$ can be expressed as an infinite sum of its derivatives evaluated at x_0 :

$$f(x_0 + \Delta x) = f(x_0) + \Delta x f'(x_0) + \frac{(\Delta x)^2}{2!} f''(x_0) + \frac{(\Delta x)^3}{3!} f'''(x_0) + \dots + \frac{(\Delta x)^m}{m!} f^{(m)}(x_0) + \dots \quad (21)$$

Using compact summation notation, this is written as:

$$f(x_0 + \Delta x) = \sum_{m=0}^{\infty} \frac{(\Delta x)^m}{m!} f^{(m)}(x_0) \quad (22)$$

In numerical analysis, this infinite series is truncated after a finite number of terms. The remaining terms are grouped together and represented using Big-O notation, $\mathcal{O}((\Delta x)^m)$, which dictates the asymptotic truncation error and the accuracy of the numerical approximation as $\Delta x \rightarrow 0$.

Temporal Derivative (Forward Difference)

To approximate the first-order temporal derivative $\frac{\partial C}{\partial t}$, we expand C forward in time ($t + \Delta t$) around the reference point (x_i, t_n) using a Taylor series expansion:

$$C_i^{n+1} = C_i^n + \Delta t \left(\frac{\partial C}{\partial t} \right)_i^n + \frac{(\Delta t)^2}{2!} \left(\frac{\partial^2 C}{\partial t^2} \right)_i^n + \mathcal{O}((\Delta t)^3)$$

We isolate the first derivative term to solve for our algebraic approximation:

$$\left(\frac{\partial C}{\partial t} \right)_i^n = \frac{C_i^{n+1} - C_i^n}{\Delta t} - \mathcal{O}(\Delta t)$$

By dropping the higher-order error terms $\mathcal{O}(\Delta t)$, we obtain the first-order accurate Forward Difference formula:

$$\left(\frac{\partial C}{\partial t} \right)_i^n \approx \frac{C_i^{n+1} - C_i^n}{\Delta t} \quad (23)$$

Spatial Derivative (Central Difference)

To approximate the second-order spatial derivative $\frac{\partial^2 C}{\partial x^2}$, we write two Taylor series expansions around the spatial reference point (x_i, t_n) : one step forward $(x + \Delta x)$ and one step backward $(x - \Delta x)$.

The forward spatial expansion is:

$$C_{i+1}^n = C_i^n + \Delta x \left(\frac{\partial C}{\partial x} \right)_i + \frac{(\Delta x)^2}{2} \left(\frac{\partial^2 C}{\partial x^2} \right)_i + \frac{(\Delta x)^3}{6} \left(\frac{\partial^3 C}{\partial x^3} \right)_i + \frac{(\Delta x)^4}{24} \left(\frac{\partial^4 C}{\partial x^4} \right)_i + \mathcal{O}((\Delta x)^5)$$

The backward spatial expansion is:

$$C_{i-1}^n = C_i^n - \Delta x \left(\frac{\partial C}{\partial x} \right)_i + \frac{(\Delta x)^2}{2} \left(\frac{\partial^2 C}{\partial x^2} \right)_i - \frac{(\Delta x)^3}{6} \left(\frac{\partial^3 C}{\partial x^3} \right)_i + \frac{(\Delta x)^4}{24} \left(\frac{\partial^4 C}{\partial x^4} \right)_i - \mathcal{O}((\Delta x)^5)$$

Adding these two equations together allows the odd-powered spatial derivative terms (Δx and $(\Delta x)^3$) to cancel out perfectly:

$$C_{i+1}^n + C_{i-1}^n = 2C_i^n + (\Delta x)^2 \left(\frac{\partial^2 C}{\partial x^2} \right)_i + \frac{(\Delta x)^4}{12} \left(\frac{\partial^4 C}{\partial x^4} \right)_i + \mathcal{O}((\Delta x)^6)$$

Isolating the second-order spatial derivative gives:

$$\left(\frac{\partial^2 C}{\partial x^2} \right)_i = \frac{C_{i+1}^n - 2C_i^n + C_{i-1}^n}{(\Delta x)^2} - \mathcal{O}((\Delta x)^2)$$

Truncating the error terms yields the second-order accurate Central Difference formula:

$$\left(\frac{\partial^2 C}{\partial x^2} \right)_i \approx \frac{C_{i+1}^n - 2C_i^n + C_{i-1}^n}{(\Delta x)^2} \quad (24)$$

Stencil Assembly

Substitute the temporal (23) and the spatial (24) derivative approximations back into (1):

$$\frac{C_i^{n+1} - C_i^n}{\Delta t} = a \frac{C_{i+1}^n - 2C_i^n + C_{i-1}^n}{(\Delta x)^2} - kC_i^n$$

Rearrange to solve for the next time step C_i^{n+1} , and we arrive at (3), the FTCS Stencil:

$$C_i^{n+1} = C_i^n + \Delta t \left[a \frac{C_{i+1}^n - 2C_i^n + C_{i-1}^n}{(\Delta x)^2} - kC_i^n \right]$$

Truncation Error

The global truncation error for this numerical scheme is quantified as $\mathcal{O}(\Delta t) + \mathcal{O}((\Delta x)^2)$.

3 Benchmark Comparison with Alternative FDM Stencils

Three Popular Explicit FDMs: FTCS, DuFort-Frankel, RK4

Benchmark: FTCS

FTCS approximates the time derivative using an first-order (explicit) forward difference, and the spatial derivative using a second-order central difference. Although a straightforward implementation, it can be limited by its stability requirements. The time step (Δt) is therefore tightly bound to the spatial dimension size (Δx).

Recall from (3), the FTCS Stencil Update Formula:

$$C_i^{n+1} = C_i^n + \Delta t \left[a \frac{C_{i+1}^n - 2C_i^n + C_{i-1}^n}{(\Delta x)^2} - k C_i^n \right] \quad (25)$$

Stability Check: Courant Number

A large decay coefficient (k) impacts the maximum allowable time step, as a fast reaction can trigger numerical underflow or artificial negative concentrations if Δt is too large. For an explicit scheme to remain stable, the Courant number must satisfy:

$$\frac{a\Delta t}{k + \Delta x^2} \leq 0.5 \quad (26)$$

DuFort-Frankel Method

DuFort-Frankel alters the standard FTCS stencil by replacing the central node value C_i^n with an average of its past and future values $(C_i^{n+1} + C_i^{n-1})/2$. Thus, it eliminates the explicit stability barrier by averaging past and future time states, thereby transforming the standard two-level explicit scheme into a three-level unconditionally stable explicit scheme. Note however that since it is a three-time-level scheme, it requires a different method (like FTCS) to calculate the very first time step. If Δt is too large, it can introduce non-physical wave errors.

Leapfrog Approximation Formula: Replaces C_i^n with a spatial average:

$$\frac{C_i^{n+1} - C_i^{n-1}}{2\Delta t} = a \frac{C_{i+1}^n - (C_i^{n+1} + C_i^{n-1}) + C_{i-1}^n}{(\Delta x)^2} - k \left(\frac{C_i^{n+1} + C_i^{n-1}}{2} \right)$$

Fourth-Order Runge-Kutta (RK4) with Explicit Time-Steps

Coupling standard central space differences with an explicit RK4 loop achieves higher temporal accuracy than standard FTCS. Instead of using a simple forward Euler step for time, the space domain is discretized using standard central differences into a system of ODEs (Method of Lines). Then, an explicit RK4 loop is used to march through time. While the jump in temporal accuracy improves tracking, the algorithm itself requires four times as many spatial derivative evaluations, significantly increasing the computational cost.

Courant Number

$$\Delta t \left(\frac{4a}{\Delta x^2} + k \right) \leq 2.785$$

Procedure: Method of Lines (MOL)

MOL is a framework where space is first discretized (using Finite Difference or Element Methods), thereby converting the PDE into a massive system of coupled ODEs. This allows for time-step off-loading to adaptive

solvers like RK4 or Backward Differentiation Formula (BDF). MOL approximates the second-order spatial derivative with central differences.

$$\frac{dC_i}{dt} = f(C_i) = a \frac{C_{i+1} - 2C_i + C_{i-1}}{(\Delta x)^2} - kC_i$$

Where $C_i(t)$ represents the time-dependent continuous concentration at the spatial node i , and $f(C_i)$ serves as the evaluation function passed to the time integrator.

RK4 Methodology

To advance the semi-discrete system from time t^n to t^{n+1} , four sequential intermediate evaluations (k_1 through k_4) are calculated across all grid points simultaneously:

1: Initial Evaluation

$$k_{1,i} = f(C_i^n) = a \frac{C_{i+1}^n - 2C_i^n + C_{i-1}^n}{(\Delta x)^2} - kC_i^n$$

2: First Midpoint Predictor

$$C_{i,1} = C_i^n + \frac{\Delta t}{2} k_{1,i}$$

$$k_{2,i} = f(C_{i,1}) = a \frac{C_{i+1,1} - 2C_{i,1} + C_{i-1,1}}{(\Delta x)^2} - kC_{i,1}$$

3: Second Midpoint Predictor

$$C_{i,2} = C_i^n + \frac{\Delta t}{2} k_{2,i}$$

$$k_{3,i} = f(C_{i,2}) = a \frac{C_{i+1,2} - 2C_{i,2} + C_{i-1,2}}{(\Delta x)^2} - kC_{i,2}$$

4: Full-Step Predictor

$$C_{i,4} = C_i^n + \Delta t k_{3,i}$$

$$k_{4,i} = f(C_{i,4}) = a \frac{C_{i+1,4} - 2C_{i,4} + C_{i-1,4}}{(\Delta x)^2} - kC_{i,4}$$

Finally, resolved using a weighted average of all four intermediate stages

$$C_i^{n+1} = C_i^n + \frac{\Delta t}{6} (k_{1,i} + 2k_{2,i} + 2k_{3,i} + k_{4,i})$$

Implicit FDMs

Implicit (backward) FDMs solve a simultaneous linear system across the whole grid at once. They are unconditionally stable and allow for large time steps Δt without exploding.

Implicit Backward-Time Central-Space (BTCS)

An implementation of this method is part of a general algorithmic comparison in Part 5. Algebraic Matrix Formula: Let $\mu = \frac{a\Delta t}{(\Delta x)^2}$. The system forms a tri-diagonal linear array:

$$-\mu C_{i-1}^{n+1} + (1 + 2\mu + k\Delta t) C_i^{n+1} - \mu C_{i+1}^{n+1} = C_i^n$$

Unconditionally stable ($\mathcal{O}(\Delta t, \Delta x^2)$).

4 A General Survey of Solvers

Besides FDMs, a wide variety of other solvers also approximate the solution to (1) with ICs (2) in a discrete, computational space.

Continuum Space Discretizations

Crank-Nicolson Method (Semi-Implicit)

Averaged Stencil Formula: Let $\mu = \frac{a\Delta t}{(\Delta x)^2}$. Trapeze-style blending creates the array:

$$-\frac{\mu}{2}C_{i-1}^{n+1} + \left(1 + \mu + \frac{k\Delta t}{2}\right)C_i^{n+1} - \frac{\mu}{2}C_{i+1}^{n+1} = \frac{\mu}{2}C_{i-1}^n + \left(1 - \mu - \frac{k\Delta t}{2}\right)C_i^n + \frac{\mu}{2}C_{i+1}^n$$

Unconditionally stable ($\mathcal{O}(\Delta t^2, \Delta x^2)$).

Finite Volume Method (FVM)

Central Differencing FVM

FVMs track mass flux balance through bounded cells rather than points. They integrate mass across control volumes bounded by faces at $i - 1/2$ and $i + 1/2$. They simplify identically to FDM for flat grids, but structurally enforce exact conservation of mass.

$$\left(\frac{C_i^{n+1} - C_i^n}{\Delta t}\right)\Delta x = \left[a\frac{C_{i+1}^n - C_i^n}{\Delta x}\right] - \left[a\frac{C_i^n - C_{i-1}^n}{\Delta x}\right] - kC_i^n\Delta x$$

Finite Element Method (FEM)

Galerkin Linear Finite Element Method

FEMs are the industry standard for handling complex, curved, or irregular physical boundaries. They project the solution onto continuous piecewise tent functions. They express the domain using tent shape functions $\phi_i(x)$. The global assembly uses a tri-diagonal Mass Matrix M and Stiffness Matrix K :

$$M = \frac{\Delta x}{6} \begin{bmatrix} 4 & 1 & 0 \\ 1 & 4 & 1 \\ 0 & 1 & 4 \end{bmatrix}, \quad K = \frac{a}{\Delta x} \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix}$$

Resulting Semi-Discrete Ordinary System:

$$M \frac{dC}{dt} + (K + kM)C = 0$$

Spectral Methods

Spectral Element Method (SEM)

SEM combines the geometric flexibility of FEM with the exponential accuracy of spectral methods. The domain is divided into a mesh of localized elements where the solution is approximated using high-order polynomials (typically 7th-Order Legendre polynomials). Spatial derivatives are computed through localized matrix-matrix multiplications.

While SEM provides exceptional spatial accuracy for smooth diffusive profiles, temporal stability is impacted. For (5), high-order spatial discretization amplifies the eigenvalues. Production solvers utilizing SEM for reaction-diffusion systems typically rely on implicit or semi-implicit time-stepping schemes to circumvent the stability bottlenecks.

Fourier Pseudo-Spectral Method

Spectral methods transform spatial values into frequency components via Forward Discrete Fourier Transforms ($\mathcal{F}$).

$$\hat{C}_m(t) = \mathcal{F}\{C(x, t)\} = \sum_{i=1}^{N_x} C_i(t) e^{-Imx_i}$$

The derivative operators are swapped entirely for multiplication scaling arrays containing wave numbers κ_m :

$$\frac{d\hat{C}_m}{dt} = (-a\kappa_m^2 - k) \hat{C}_m \implies \hat{C}_m^{n+1} = \hat{C}_m^n e^{(-a\kappa_m^2 - k)\Delta t}$$

Values map back using the Inverse Fourier Transform: $C_i^{n+1} = \mathcal{F}^{-1}\{\hat{C}_m^{n+1}\}$.

Probabilistic/Kinetic Frameworks

Lattice Boltzmann Method (LBM)

Unlike continuum methods that discretize (5) directly, LBM tracks the evolution of fictitious particle distribution functions $f_\alpha(x, t)$ along discrete lattice velocity directions $\mathbf{e}_\alpha$. The concentration field $C(x, t)$ emerges as a local hydrodynamic moment: $C = \sum_\alpha f_\alpha$. The method is completely local in its streaming step, making it exceptionally fast for parallel computing (GPUs).

The governing Boltzmann equation utilizes a consecutive "collide and stream" sequence. Using the standard BGK (Bhatnagar-Gross-Krook) single-relaxation-time collision operator, the update rule is:

$$f_\alpha(x + \mathbf{e}_\alpha \Delta t, t + \Delta t) = f_\alpha(x, t) - \frac{1}{\tau} [f_\alpha(x, t) - f_\alpha^{eq}(x, t)] + \Delta t w_\alpha R$$

Here, τ is the dimensionless relaxation time linked to the diffusion coefficient ($a = c_s^2(\tau - 0.5)\Delta t$), f_α^{eq} is the local equilibrium distribution function, w_α is the lattice weighting factor, and $R = -kC$ handles the linear sink reaction term inside the collision step. Through a multiscale expansion, (1) is recovered exactly with second-order spatial and temporal accuracy ($\mathcal{O}(\Delta t^2, \Delta x^2)$).

Stochastic Particle Tracking (Random Walk / Monte Carlo)

Instead of treating the concentration as a continuous field, this models a discrete population of individual particles undergoing Brownian motion (representing diffusion) alongside stochastic death/birth rules (representing reaction).

Alternative Spatial Discretizations

Discontinuous Galerkin (DG) Method

A hybrid approach inheriting the high-order accuracy of FEMs and the strict, localized physical mass conservation of FVMs.

Meshless Radial Basis Functions (RBF)

These skip traditional coordinate grids entirely, using scattered node points and radial basis functions to calculate derivatives, making them excellent for highly irregular geometries.

Machine Learning Approaches

Physics-Informed Neural Networks (PINN)

A mesh-free technique where a deep neural network approximates the concentration function $C(x, t)$. The residual of (1) is directly embedded into the network's loss function to guide optimization.

Fourier Neural Operators (FNO)

FNO learns the underlying macroscopic operator mapping between infinite-dimensional function spaces (e.g., mapping the initial condition $C(x, 0)$ directly to the solution field $C(x, t)$), bypassing traditional spatial grids by performing non-local convolutions in the frequency domain using the Fast Fourier Transform ($\mathcal{F}$).

$$v^{(l+1)}(x) = \sigma \left(W v^{(l)}(x) + \mathcal{F}^{-1} \left\{ R \cdot \mathcal{F}\{v^{(l)}(x)\} \right\} \right)$$

Here, $v^{(l)}$ represents the neural network’s hidden representation at layer l , W is a linear pixel-wise transformation, R is a matrix of learnable complex weights that parameterize the kernel operator in the frequency domain, and σ is a non-linear activation function.

For reaction-diffusion equations, FNO offers zero-shot super-resolution, allowing for training on coarse, low-resolution simulations and evaluated on arbitrarily fine grids with zero loss in accuracy. FNO is mesh-independent, and once trained, evaluates much faster than traditional numerical solvers. However, FNO lacks strict physical enforcement guarantees unless coupled with a physics-informed loss term.

Advanced Techniques

Total Variation Diminishing (TVD) and Flux Limiters

Standard higher-order central space differences suffer from dispersion when encountering the discontinuous edges of (2). This triggers oscillations (the Gibbs phenomenon) and artificial negative concentrations. High-Resolution Schemes introduce non-linear flux limiters $\phi(r)$ to switch between low-order upwind and high-order central approximations, based on local gradients. The cell interface flux is modified as:

$$F_{i+1/2} = F_{i+1/2}^{\text{Low}} + \phi(r_i) \left(F_{i+1/2}^{\text{High}} - F_{i+1/2}^{\text{Low}} \right)$$

Here, r_i represents the ratio of consecutive concentration gradients. By enforcing a Total Variation Diminishing (TVD) constraint, limiters ensure that the solver maintains non-oscillatory profiles at $|x| = 1$ without spreading or generating artificial extrema.

Adaptive Mesh Refinement (AMR)

The ICs (2) feature high-gradient fronts at $x = -1$ and $x = 1$, while the remainder of the spatial domain stays entirely flat. Utilizing a uniform grid across the entire domain is inefficient. AMR dynamically creates localized, fine-grained sub-grids around these fronts while maintaining a coarse mesh in smooth regions. The refinement criteria rely on a local truncation error estimate or the gradient of the concentration:

$$\chi_i = \left| \frac{\partial^2 C}{\partial x^2} \right| > \epsilon_{\text{threshold}}$$

When χ_i exceeds the user-defined threshold, the cell is automatically subdivided. As the diffusion front smooths out and propagates over time, the sub-grids dynamically transform, reducing overhead.

Operator Splitting Methods: Godunov Time-Splitting

Godunov Time-Splitting (Explicit) breaks each time step into two sequential, purely explicit sub-steps. First, a pure diffusion equation over a half-step using explicit central differences. Then, the results are used to solve a pure reaction/decay equation over the next half-step. It decouples the competing physical phenomena, which can make debugging individual terms much simpler, but also introduces a splitting error that scales with the time step.

Advanced Time Integrators: Exponential Time Differencing (ETD) / Integration Factor (IF)

These methods evaluate the linear diffusion term exactly using matrix exponential, lifting the harsh diffusion stability barrier while treating the reaction term explicitly or semi-implicitly.

Comparative Matrix of Solvers of (1) with ICs (2)

Category	Method	Accuracy & Stability	Handling of Sharp Discontinuities ($ x = 1$)	Primary Engineering Trade-off
Continuum	Crank-Nicolson	$\mathcal{O}(\Delta t^2, \Delta x^2)$ / Unconditional	Severe oscillations (Gibbs phenomenon)	Balanced baseline but requires implicit global matrix solves.
	Finite Volume Method (FVM)	$\mathcal{O}(\Delta t, \Delta x^2)$ / Explicit Limit	Preserves local mass; smears sharp edges	Exact physical conservation vs. first-order time step limit.
	Galerkin Linear Finite Element Method (FEM)	$\mathcal{O}(\Delta t, \Delta x^2)$ / Scheme Dependent	Triggers localized node-to-node oscillations	Geometric boundary flexibility vs. high spatial stiffness.
Spectral Methods	Spectral Element Method (SEM)	$\mathcal{O}(\Delta t^p, \Delta x^N)$ / Stiff Explicit	Amplifies high-frequency spatial noise at edges	Exponential smooth accuracy vs. severe eigenvalue restriction.
	Fourier Pseudo-Spectral	$\mathcal{O}(\Delta t, e^{-N})$ / Unconditional	Global ringing; spreads error across entire domain	Good for smooth profiles; unsuited for shocks.
Probabilistic / Kinetic Frameworks	Lattice Boltzmann Method (LBM)	$\mathcal{O}(\Delta t^2, \Delta x^2)$ / Local Stability	Resolves fronts smoothly without statistical noise	Fast GPU parallelization vs. rigid lattice geometry.
	Stochastic Tracking	$\mathcal{O}(\Delta t^{1/2})$ / Mesh-Free	Preserves sharp boundaries naturally; zero wiggles	High mesh-free flexibility vs. statistical noise overhead.
Alternative Spatial Discretizations	Discontinuous Galerkin	$\mathcal{O}(\Delta t^p, \Delta x^N)$ / Explicit Limit	Captures interface fluxes cleanly; localized wiggles	High-order sub-element accuracy vs. degree-of-freedom cost.
	Meshless Radial Basis Functions (RBF)	Variable / Node Dependent	Sensitive to matrix ill-conditioning near steps	Skips grid-generation entirely vs. dense, expensive matrix inversion.
Machine Learning Approaches	Physics Informed Neural Network (PINN)	Mesh-Free / N/A	Struggles to locate and fit sharp localized steps	Eliminates spatial meshing vs. unpredictable optimization convergence.
	Fourier Neural Operator (FNO)	Operator Level / Infinite-Dim	Evaluates sharp fronts with zero-shot resolution	Near-instant future inference vs. massive data training requirements.
Advanced Techniques	Godunov Splitting	$\mathcal{O}(\Delta t)$ Temporal / Split Limit	Decouples diffusion wiggles from reaction step	Simplifies complex multi-physics vs. permanent splitting error.
	Exponential Integrators	Higher-Order / Unconditional	Captures diffusion smoothly; reaction dependent	Eliminates stiff linear time limits vs. matrix exponential cost.
	AMR + TVD / Flux Limiters	Adaptive / Non-Linear	Dynamically suppresses all wiggles; sharp step tracks	Perfect front tracking vs. complex data structure/interface logic.

5 FDM Performance Comparison with Python

fdsolvers.py

```
import numpy as np
from scipy.special import erf
from scipy.linalg import solve_banded

# boiler-plate
def analytical_solution(x, t, a, k):
    """
        Analytic Solution to Reaction Diffusion/Heat Equation
        with Top-Hat Initial Conditions at  $|x| < 1$ 
    """
    if np.isclose(t, 0.0): # why am i doing this?
        C = np.where(np.abs(x) < 1.0, 1.0, 0.0)
        C[np.isclose(np.abs(x), 1.0)] = 0.5
        return C
    denom = 2.0 * np.sqrt(a * t)
    erf_plus = erf((1.0 + x) / denom)
    erf_minus = erf((1.0 - x) / denom)
    C = 0.5 * np.exp(-k * t) * (erf_plus + erf_minus)
    return C

def ftcs(x, dx, dt, t_final, a, k):
    """
        Forward-Time Central-Space (FTCS) Method
    """
    nx = len(x)
    nt = int(round(t_final / dt))
    C = np.where(np.abs(x) < 1.0, 1.0, 0.0)
    C[np.isclose(np.abs(x), 1.0)] = 0.5
    r = a * dt / (dx**2)
    for n in range(nt):
        C_next = np.copy(C)
        C_next[1:-1] = C[1:-1] +
            r * (C[2:] - 2*C[1:-1] + C[:-2]) - k * dt * C[1:-1]
        C_next[0] = 0.0
        C_next[-1] = 0.0
        C = C_next
    return C

def rk4_derivative(C, r_factor, k_dt):
    """
        Runge-Kutta 4 Spatial Derivative
    """
    dC_dt = np.zeros_like(C)
    dC_dt[1:-1] = r_factor * (C[2:] -
        2.0 * C[1:-1] + C[:-2]) - k_dt * C[1:-1]
    dC_dt[0] = 0.0
    dC_dt[-1] = 0.0
    return dC_dt

def rk4(x, dx, dt, t_final, a, k):
    """
        Runge-Kutta 4 Main Algorithm
    """
    nx = len(x)
    nt = int(round(t_final / dt))
```

```

C = np.where(np.abs(x) < 1.0, 1.0, 0.0)
C[np.isclose(np.abs(x), 1.0)] = 0.5
r_factor = a * dt / (dx**2)
k_dt = k * dt
for _ in range(nt):
    k1 = rk4_derivative(C, r_factor, k_dt)
    k2 = rk4_derivative(C + 0.5 * k1, r_factor, k_dt)
    k3 = rk4_derivative(C + 0.5 * k2, r_factor, k_dt)
    k4 = rk4_derivative(C + k3, r_factor, k_dt)
    C += (k1 + 2.0 * k2 + 2.0 * k3 + k4) / 6.0
return C

def dufort frankel(x, dx, dt, t_final, a, k):
    nx = len(x)
    nt = int(round(t_final / dt))
    C_past = np.where(np.abs(x) < 1.0, 1.0, 0.0)
    C_curr = np.copy(C_past)
    r = a * dt / (dx**2)
    C_curr[1:-1] = C_past[1:-1] + r * (C_past[2:] -
        2*C_past[1:-1] + C_past[:-2]) - k * dt * C_past[1:-1]
    denom = 1.0 + 2.0 * r + k * dt
    alpha = (1.0 - 2.0 * r - k * dt) / denom
    beta = (2.0 * r) / denom
    for n in range(1, nt):
        C_next = np.zeros(nx)
        C_next[1:-1] = (alpha * C_past[1:-1] +
            beta * (C_curr[2:] + C_curr[:-2]))
        C_next[0] = 0.0
        C_next[-1] = 0.0
        C_past = np.copy(C_curr)
        C_curr = C_next
    return C_curr

def htcs(x, dx, dt, t_final, a, k):
    """
    (Implicit) Backward-Time Central-Space Method
    """
    nx = len(x)
    nt = int(round(t_final / dt))
    C = np.where(np.abs(x) < 1.0, 1.0, 0.0)
    C[np.isclose(np.abs(x), 1.0)] = 0.5
    r = a * dt / (dx**2)
    diag_main = (1.0 + 2.0 * r + k * dt) * np.ones(nx - 2)
    diag_sub = -r * np.ones(nx - 2)
    diag_super = -r * np.ones(nx - 2)
    A_banded = np.zeros((3, nx - 2))
    A_banded[0, 1:] = diag_super[:-1]
    A_banded[1, :] = diag_main
    A_banded[2, :-1] = diag_sub[1:]
    for n in range(nt):
        B = np.copy(C[1:-1])
        B[0] += r * 0.0
        B[-1] += r * 0.0
        C[1:-1] = solve_banded((1, 1), A_banded, B)
        C[0] = 0.0
        C[-1] = 0.0
    return C

```

Report Github Repository Code-base

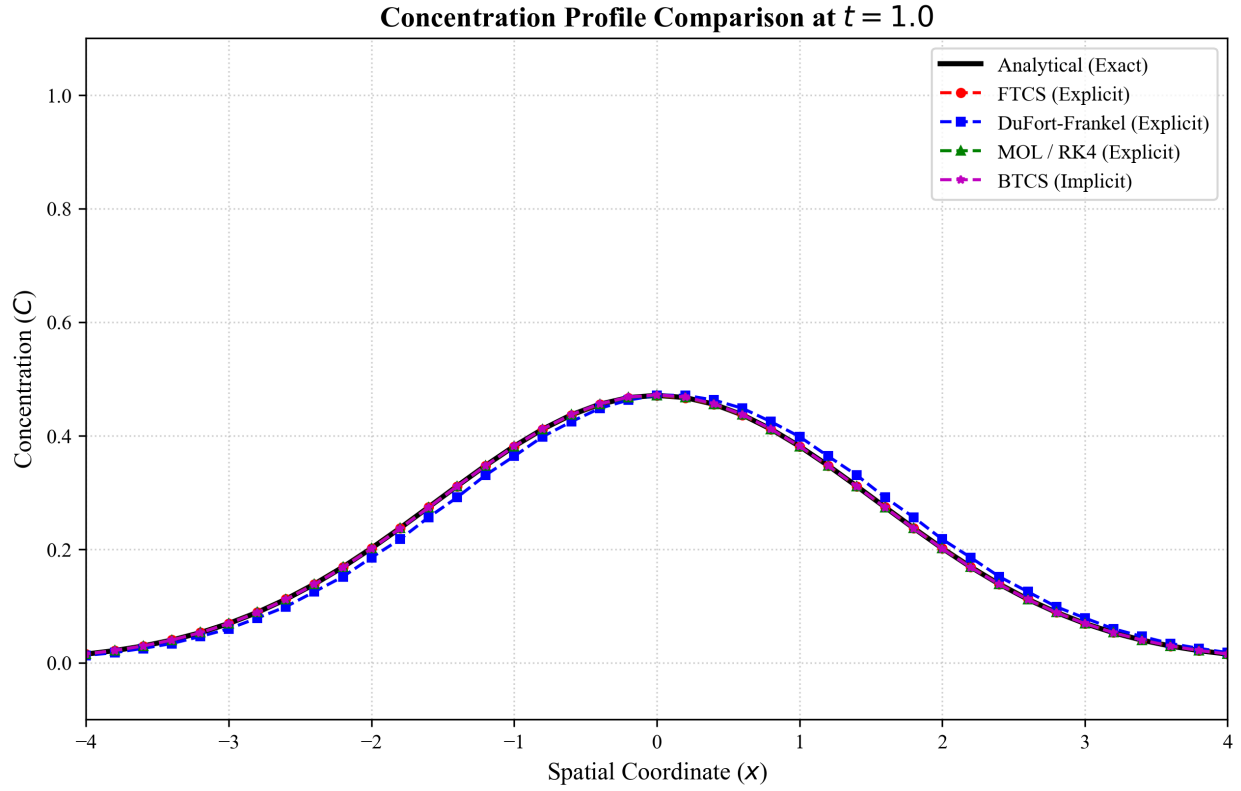


Figure 1: Enter Caption

Plotting Notes

Show the physical effects of a and k :

- Effect of a (Diffusion Coefficient): Higher values of a flatten out the concentration pulse faster over space (x).
- Effect of k (Reaction/Decay Coefficient): Higher values of k exponentially decrease the peak amplitude over time (t) without widening the spread.